Flask-Based Student Registration Application with CI/CD Deployment

This project is a simple student registration web application built with Python Flask and MySQL. It is deployed on AWS EC2 and uses Jenkins for automated CI/CD so that every time you push code to GitHub, the application gets updated automatically without any manual work.

What This Project Does

- Students can fill out a registration form with their name, email, and course.
- The data is saved into a MySQL database.
- Jenkins watches the GitHub repository, and whenever new code is pushed, it pulls the latest code, installs dependencies, and restarts the Flask application automatically.

Architecture

The project uses three separate EC2 servers, each handling one responsibility.

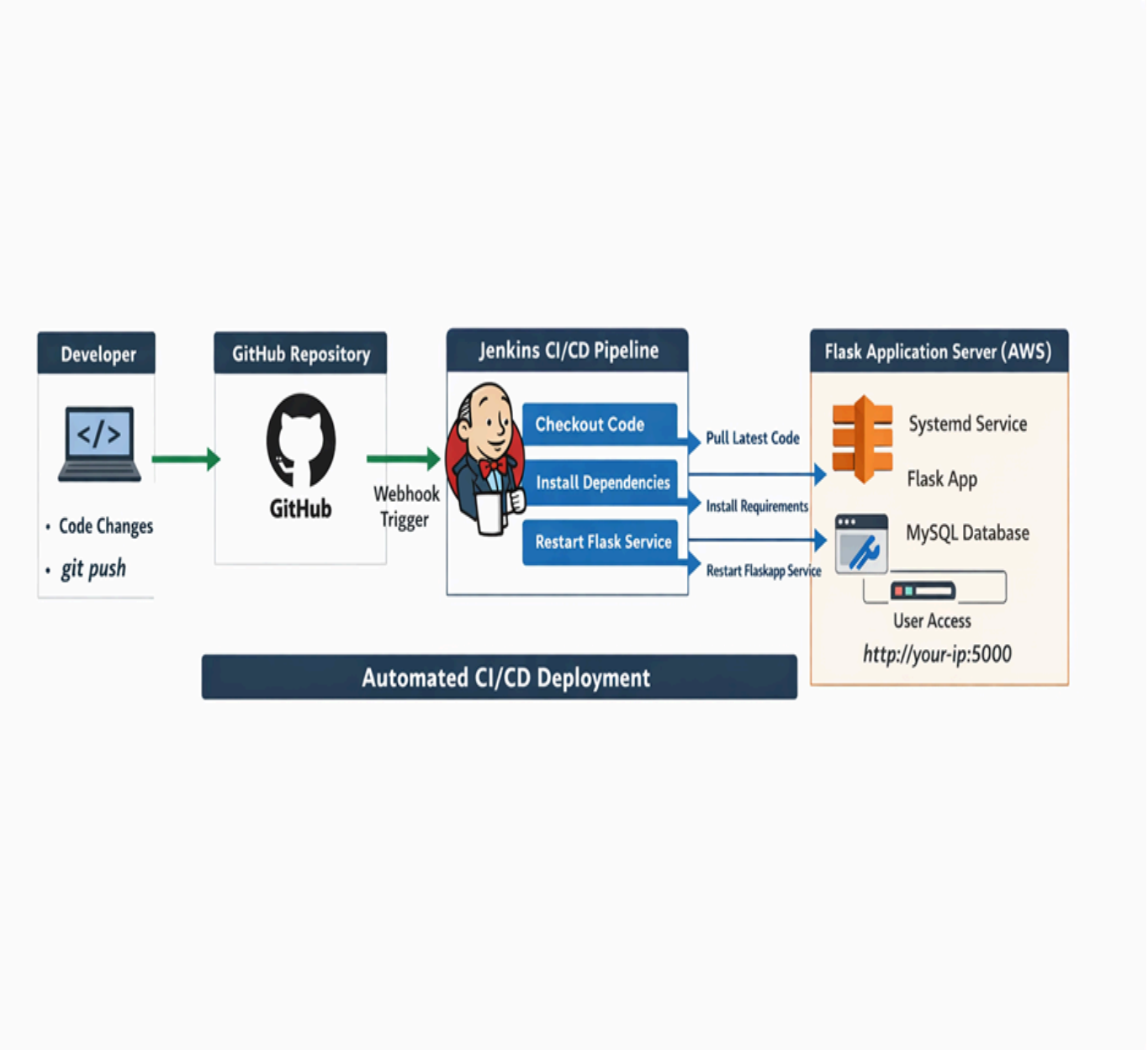
Jenkins Server handles CI/CD. Whenever code is pushed to GitHub, a webhook triggers Jenkins, which runs the pipeline to deploy the updated application.

Flask Application Server runs the Python Flask web application. It connects to the MySQL database to store and retrieve student records.

MySQL Database Server stores all student registration data in a database called `studentdb`.

The flow looks like this:

```
Developer pushes code to GitHub
        |
GitHub sends webhook to Jenkins
        |
Jenkins pulls latest code on Flask server
        |
Jenkins installs dependencies and restarts Flask app
        |
Updated application is live
```



Technologies Used

- Python Flask
- MySQL (MariaDB on Amazon Linux)
- Git and GitHub
- Jenkins
- AWS EC2 (Amazon Linux 2)

Setup Instructions

Step 1 - Create EC2 Instances

Create three EC2 instances on AWS with the following settings:

Jenkins Server

- AMI: Amazon Linux 2

- Instance Type: t3.micro
- Storage: 10 GB
- Open port 8080 in the security group for Jenkins UI

Flask App Server

- AMI: Amazon Linux 2
- Instance Type: t2.micro
- Storage: 10 GB
- Open port 22 (SSH), port 5000 (Flask), and port 80 (HTTP)

MySQL Database Server

- AMI: Amazon Linux 2
- Instance Type: t2.micro
- Storage: 10 GB
- Open port 22 (SSH) and port 3306 (MySQL, accessible from the Flask server only

---

Step 2 - Setup MySQL Database Server

SSH into the database server and run the following commands:

```
sudo yum update -y
sudo yum install mariadb105-server -y
sudo systemctl start mariadb
sudo systemctl enable mariadb
```

Log into MySQL and set up the database:

```
mysql -u root -p

CREATE DATABASE studentdb;
USE studentdb;

CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  course VARCHAR(100)
);

CREATE USER 'flaskuser'@'%' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON studentdb.* TO 'flaskuser'@'%';
FLUSH PRIVILEGES;
```

Step 3 - Setup Flask Application Server

SSH into the Flask server and install the required packages:

```
sudo yum update -y
sudo yum install python3 git -y
```

Clone your application code and install Python dependencies:

```
git clone <your-repo-url>
cd student-registration-application
pip3 install -r requirements.txt
```

Configure the Flask app as a systemd service so it runs in the background and restarts automatically:

```
sudo nano /etc/systemd/system/flaskapp.service
```

Paste this configuration:

```
[Unit]
Description=Flask Application
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/workspace/jenkins-pipeline/student-registration-application
ExecStart=/usr/bin/python3 /home/ec2-user/workspace/jenkins-pipeline/student-registration-application/app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable flaskapp
sudo systemctl start flaskapp
```

---

access the web page using http://<public_ip>:5000

Step 4 - Setup Jenkins Server

SSH into the Jenkins server and install Java and Jenkins:

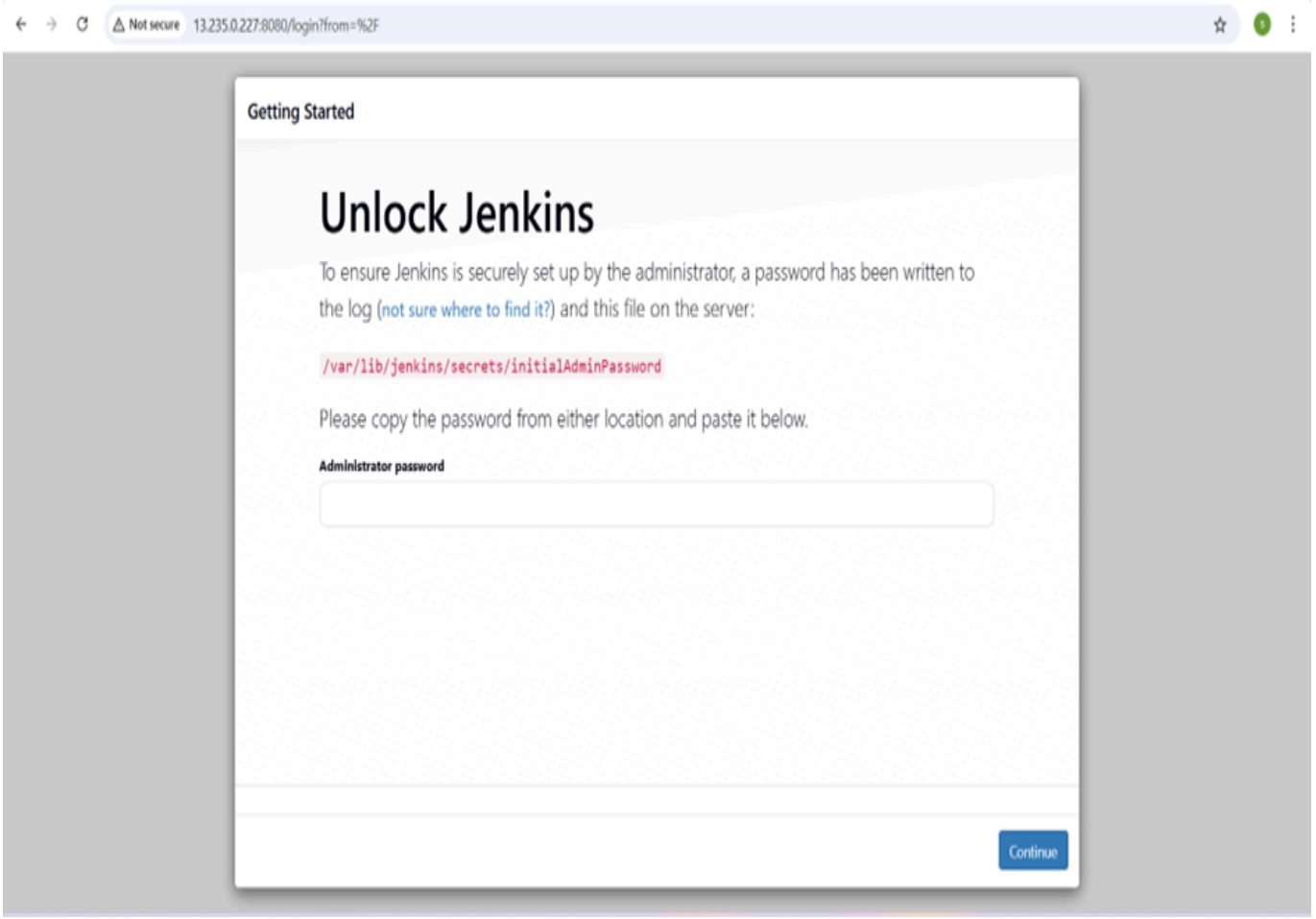
```
sudo yum install java-17-amazon-corretto -y
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/rpm-stable/jenkins.repo
sudo yum upgrade
sudo yum install jenkins -y
sudo yum install git -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

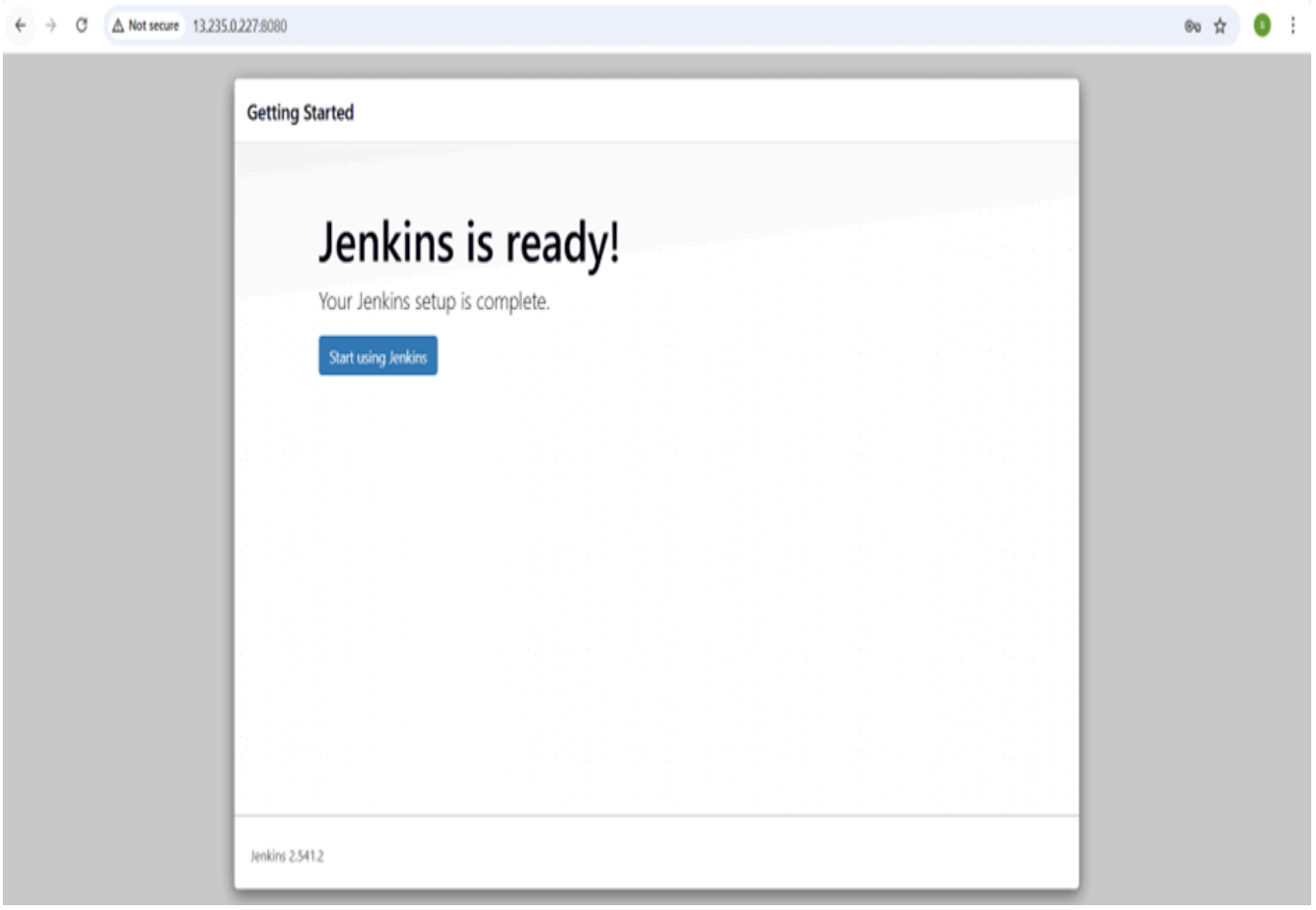
Open Jenkins in your browser at `http://<JENKINS_PUBLIC_IP>:8080` and complete the setup using the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Install these plugins from Manage Jenkins > Plugins > Available Plugins:

- Git Plugin
- GitHub Plugin
- Pipeline Plugin
- SSH Agent Plugin





Step 5 - Push Code to GitHub

On the Flask server, push your application code to GitHub:

```
cd student-registration-application
git init
git add .
git commit -m "initial commit"
git remote add origin <your-github-repo-url>
git push origin master
```

```
student-registration-app
│
├── app.py
├── requirements.txt
├── Jenkinsfile
├── README.md
│
├── templates
│   ├── index.html
│   └── students.html
│
└── static
    └── style.css
```

Step 6 - Create Jenkins Pipeline

1. Go to Jenkins Dashboard and click New Item.
2. Name it `flask-pipeline`, select Pipeline, and click OK.
3. Under Pipeline settings, set Definition to "Pipeline script from SCM".
4. Set SCM to Git and enter your repository URL.
5. Set branch to `*/master` and Script Path to `Jenkinsfile`.
6. Click Save.

Add a `Jenkinsfile` in your GitHub repository with the following content:

```
pipeline {
    agent { label 'flask' }
    stages {
        stage('Checkout Code') {
            steps {
                git 'https://github.com/DJSACHIN9511/Flask-Based-Student-
Registration-Application-with-CI-CD-Deployment.git'
            }
        }
        stage('Install Dependencies') {
```

```
            steps {
                sh '''
                cd student-registration-application
                pip3 install -r requirements.txt
                '''
            }
        }
        stage('Restart Flask Service') {
            steps {
                sh '''
                sudo systemctl restart flaskapp
                '''
            }
        }
    }
}
```

Also allow Jenkins to restart services without a password prompt:

```
sudo visudo
```

Add this line at the end:

```
ec2-user ALL=(ALL) NOPASSWD: ALL
```

Step 7 - Setup GitHub Webhook

1. Go to your GitHub repository Settings > Webhooks > Add Webhook.
2. Set the Payload URL to `http://<JENKINS_PUBLIC_IP>:8080/github-webhook/`
3. Set Content type to `application/json`.
4. Save the webhook.

Now every time you push code to GitHub, Jenkins will automatically deploy the updated application.

How to Test

1. Open the Flask app in your browser at `http://<FLASK_SERVER_IP>:5000`
2. Fill in the student registration form and submit.
3. Log into the database server and check that the record was saved:

```
mysql -u flaskuser -p
USE studentdb;
SELECT * FROM students;
```

Challenges Faced

Connecting Jenkins to the Flask Server was tricky at first. Jenkins needs to SSH into the Flask server to run deployment commands. Setting up the SSH agent and making sure the correct key permissions were in place took some trial and error.

Passwordless sudo for Jenkins was a required step that is easy to miss. Without it, the pipeline fails when trying to restart the Flask systemd service because Jenkins does not have sudo privileges by default.

MySQL remote access required making sure the database server's security group allowed connections from the Flask server's IP on port 3306. Initially the connection was being refused because of this.

Systemd service working directory had to exactly match where Jenkins was checking out the code. If the path was slightly wrong, the Flask app would fail to start after deployment.

Future Improvements

- Add HTTPS using an SSL certificate so the application runs securely instead of over plain HTTP.
- Use Docker to containerize the Flask application so it is easier to deploy consistently across different environments.
- Add a staging environment where code is first deployed and tested before going to production.
- Store database credentials in environment variables or AWS Secrets Manager instead of hardcoding them in the application.
- Add automated tests in the Jenkins pipeline so that broken code never gets deployed.
- Set up monitoring and alerts using a tool like CloudWatch so you know immediately if the application goes down.
- Use an RDS instance instead of a self-managed MySQL server for better reliability and automated backups.

Repository Structure

```
student-registration-application/
    app.py
    requirements.txt
    templates/
        index.html
Jenkinsfile
README.md
```

Author

Sachin Jaiswal